

Lab 8: Memory Ordering Lab

ID : 12532588 NAME : Li Zihao

1. Question 1: Store-Load Reordering Without Fence

2.1 Command

```
cd order_lab/Q1
make run
```

2.2 Output Record

- Number of violations: 38
- Percentage of violations: 19%
- Total trials: 200

2.3 make run-inorder or Equivalent In-Order Test

Command / Makefile Change

Q1 的 Makefile 没有单独提供 run-inorder target , 所以这里按题面要求改 FLAGS 为 --dual --inorder --bpred -b 后重新执行 make run 。

```
cd order_lab/Q1
# 将 Makefile 里的 FLAGS 从 --dual --ooo --bpred -b
# 改成 --dual --inorder --bpred -b
make run
```

Result

- Violations under in-order : 0
- Percentage under in-order : 0%
- Total trials under in-order : 200

2.4 Analysis

第一次运行 `make run` 时，Q1 在 200 次试验中观察到 38 次 violation，比例为 19%。这说明在当前 `dual-core + 000 + TS0 + per-core store buffer` 的实验环境下，`r1 = 0 && r2 = 0` 不是偶发到几乎看不见的结果，而是能够被稳定观察到的现象。随后按题面把配置改成 `--dual -inorder --bpred -b` 后再次运行，200 次试验中的 violation 变成 0，说明这类双零结果并不是只要有 TS0 就一定会出现，而是要靠更积极的执行机制把它真正暴露出来。

2.5 Why 000 + TS0 Makes Violations Possible

TS0 允许 Store-Load Reordering，也就是前面的 store 还没有从每个 core 自己的 store buffer 对外可见时，后面的 load 可以先被观察到；000 core 又会更积极地让后续已经满足条件的 load 提前执行。因此，两边的 store 都可能暂时停留在各自的 store buffer 里，而两边的 load 已经先去共享层读取旧值，于是就会出现 `r1 = 0 && r2 = 0` 这种在 SC 下本来不该出现的结果。相对地，in-order core 大体按程序顺序推进，不会主动把后面的 load 穿过前面的 store 提前执行，所以在同样的 litmus test 下观察到 0 次 violation。

2. Question 2: Fence Eliminates the Double-Zero Outcome

3.1 Command

```
cd order_lab/Q1
make run-fence
```

3.2 Result

- Violations after inserting `fence rw, rw: 0`
- Percentage after inserting `fence rw, rw: 0%`
- Total trials: 200

3.3 Explanation

在 Q1 里，每个 core 的程序顺序本来都是“先 store，再 load”。问题在于 TS0 允许 store 暂时停在本 core 的 store buffer 里，还没对另一个 core 可见时，后面的 load 就先执行，所以才会出现两边都读到旧值 0 的情况。

加入 `fence rw, rw` 之后，store 和后面的 load 之间建立了强顺序约束：栅栏前的读写没有完成并对外可见之前，栅栏后的读写不能越过它执行。放到这道题里，就是：

1. Core 0 的 `x = 1` 必须先排空 store buffer、变成全局可见，Core 0 才能执行 `r1 = y`
2. Core 1 的 `y = 1` 必须先排空 store buffer、变成全局可见，Core 1 才能执行 `r2 = x`

因此，如果 Core 0 真的读到了 `y = 0`，那就说明 Core 1 的 `y = 1` 还没有完成 fence 前的可见化，此时 Core 1 还不能继续执行自己的 `load x`；反过来也一样。于是两个 core 不可能同时都已经执行完自己的 `load`，同时又都只看到旧值 `0`。所以 `r1 = 0 && r2 = 0` 被消除了。

3.4 Happens-Before Timeline

Core 0 store x = 1 v fence rw, rw waits until x is globally visible v load r1 = y	Core 1 store y = 1 v fence rw, rw waits until y is globally visible v load r2 = x
---	---

Program-order / happens-before edges:

`x = 1` -> fence0 -> `r1 = y`

`y = 1` -> fence1 -> `r2 = x`

Key consequence:

before Core 0 can do `r1 = y`, `x = 1` must already be visible

before Core 1 can do `r2 = x`, `y = 1` must already be visible

So the two loads cannot both run while both stores are still invisible.

Therefore `r1 = 0 && r2 = 0` is impossible.

3. Question 3: Lost Updates in `order_lab/Q2`

4.1 Command

```
cd order_lab/Q2
make run
```

4.2 Output Record

- Actual counter value: `1000`
- Expected value: `2000`
- Lost updates: `1000`

4.3 Calculation

```
lost updates = 2000 - actual counter value
              = 2000 - 1000
```

4.4 Analysis

这道题里的 `counter++` 并不是原子操作，而是一个 `read-modify-write` 序列：

```
tmp = *counter;
tmp = tmp + 1;
*counter = tmp;
```

问题就在于两个 core 可能同时读到同一个旧值 `k`。例如，Core 0 先读到 `k`，算出 `k+1`，但它的 `store` 先进入自己的 `store buffer`，还没有立刻对 Core 1 可见；这时 Core 1 也仍然可能从共享层读到旧值 `k`，同样算出 `k+1`，最后两个 core 都把 `k+1` 写回去。这样本来应该增加两次，结果只增加了一次，于是就丢掉了 `一个 update`。

这次实测里每个 core 做 `1000` 次递增，理论值应为 `2000`，但实际只得到 `1000`。这说明大量递增都发生了“两个 core 读同一个旧值，再互相覆盖”的情况，所以最终结果退化成了大约只保留其中一个 core 的有效递增。

4.5 Why the Loss Count Is Close to `N_ITER`

题面已经明确两个关键条件：

1. 每个 core 都有自己的 `store buffer`
2. `store buffer` 里的已提交写入不会立刻对另一个 core 可见，而是逐步 drain 到共享 L2

在此基础上，从当前模拟器实现还能看到两点会进一步放大这个现象：

1. `STORE_BUF_SIZE = 16`
2. 双核模式下 `store buffer` 的 drain 会按 `LAT_L2_HIT = 8` 的节奏推进，也就是一个条目要隔若干周期才能真正写到共享层

这意味着两个 core 的递增循环跑得很快，但每次写回 `counter` 的新值传播得相对慢。于是常见情况不是“偶尔几次碰撞”，而是：

1. 两边反复从共享层看到同一个较旧的 `counter`
2. 两边分别在本地算出同样的新值
3. 两边又把同样的新值放进各自的 `store buffer`
4. 最后 drain 出去时，很多轮都只是把同一个值重复写回

所以每一轮双核并发递增，往往只真正留下一个有效增量，另一个增量被覆盖掉。总共有两个 core、每个 core `1000` 次递增，最后结果就自然会接近 `1000`，也就是丢失量接近 `N_ITER`，而

不是只损失一个很小的零头。

4. Question 4: Broken Peterson Mutex Under TSO

5.1 Command

```
cd order_lab/Q3
make run
```

5.2 Output Record

- Actual counter value: 2000
- Expected counter value: 4000

5.3 Which Step of `lock()` Fails Under TSO

失效的不是 Peterson 算法的数学逻辑本身，而是 `lock()` 里进入自旋判断前所依赖的可见性假设。在 `sc` 下，执行：

```
want[i] = 1;
*turn = j;
while (want[j] && *turn == j) {}
```

时，另一个 core 应该已经能看到 `want[i] = 1`。但在 `TSO` 下，`want[i] = 1` 可能还停留在本 core 的 `store buffer` 里，没有及时对对方可见。于是对方在执行 `while (want[j] && *turn == j)` 时，仍然可能读到陈旧的 `want[j] == 0`，错误地认为“没人和我竞争”，直接跳出自旋并进入临界区。

所以，真正失败的是 `while (want[j] && *turn == j)` 这一步对前面两次写入可见性的依赖：在 `TSO` 下，没有 `fence` 保证时，这个判断会基于旧值作出错误结论。

5.4 Failure Timeline

```
Core 0
want[0] = 1    (stays in store buffer)
turn = 1
load want[1] -> still sees 0
while (want[1] && turn == 1) exits
enter critical section
counter++ overlaps with Core 1
```

```
Core 1
want[1] = 1    (stays in store buffer)
turn = 0
load want[0] -> still sees 0
while (want[0] && turn == 0) exits
enter critical section
counter++ overlaps with Core 0
```

Broken reasoning:

```
each core thinks the other core does not want to enter
because each load reads a stale cached 0 instead of the other core's buffered want[] store
```

```
Result:
both cores pass the spin-wait at the same time
mutual exclusion is violated
the protected counter increment loses updates again
```

5.5 Analysis

这次程序本来想用 Peterson 锁把共享 `counter` 保护起来，所以理论上两个 core 各执行 2000 次递增后，最终值应该是 4000。但实测只有 2000，说明临界区并没有真正做到“同一时刻只允许一个 core 进入”。

根本原因是：Peterson 锁依赖共享变量 `want[]` 和 `turn` 的顺序可见性，而 TS0 下每个 core 都有自己的 store buffer。当 Core 0 写 `want[0] = 1` 时，这个值可能暂时还没有对 Core 1 可见；Core 1 写 `want[1] = 1` 也是同理。这样两边在读 `want[j]` 时，都可能错误地看到 0，从而同时跳出 `while`，一起进入临界区。

一旦两个 core 同时进入临界区，里面的 `counter++` 就又退化成了前面 Q3 里的非原子 `read-modify-write`，于是 lost updates 再次出现。现在的结果 4000 -> 2000，说明这把 broken Peterson 锁在当前 TS0 + 000 + per-core store buffer 环境下并没有起到互斥保护作用。

5. Question 5: Fixing Peterson with Two Fences

6.1 Modified Code Snippet

```
static void lock(int i) {
    int j = 1 - i;
    want[i] = 1;
    __asm__ volatile("fence rw, rw");
    *turn = j;
    __asm__ volatile("fence rw, rw");
    while (want[j] && *turn == j) {}
}
```

6.2 Verification Command

```
cd order_lab/Q3
make run-fence
```

6.3 Verification Result

虽然 lab8.pdf 在 Question 5 的最后一句写成了“verify the counter equals `N_ITER`”，但同一题前文已经说明两边都会各自执行 `N_ITER` 次递增；实际 benchmark 源码和模拟器输出也都使用 `2 * N_ITER` 作为正确结果。因此这里按源码与实测记录 `Expected = 4000`。

- Actual counter value: `4000`
- Expected counter value: `4000`

6.4 Why Two Fences Are Required

这两道 fence 不是重复写法，而是在修两个不同的顺序漏洞。Peterson 锁要正确，进入 `while (want[j] && *turn == j)` 之前，当前 core 对共享内存做的两次关键写入都必须已经按正确顺序对另一个 core 可见：

1. `want[i] = 1`
2. `turn = j`

如果其中任何一次写入还停在本 core 的 `store buffer` 里，或者被后面的读条件“抢跑”，另一个 core 就可能基于旧值作出错误判断，导致两个 core 同时通过 spin-wait。

6.5 What Reordering the First Fence Prevents

第一道 fence 放在 `want[i] = 1` 后面，作用是先把“我要进入临界区”这个声明推出去，再允许后续逻辑继续。

它要防的是这类错误顺序：

```
want[i] = 1    还没对外可见
load want[j]   已经开始读对方状态
```

如果没有这道 fence，当前 core 可能在自己的 `want[i]` 还没被别人看到时，就已经开始检查 `want[j]`。这会让双方都觉得“对面还没想进来”，于是双双跳出 `while`。所以第一道 fence 解决的是 `want[i]` 的 store-to-load 可见性问题。

6.6 What Reordering the Second Fence Prevents

第二道 fence 放在 `turn = j` 后面，作用是先把“这轮优先让给对方”这个决定写稳，再允许进入 `while` 的读取阶段。

它要防的是这类错误顺序：

```
turn = j      还没对外可见
```

```
load want[j] / load turn 已经开始参与自旋判断
```

也就是说，如果没有第二道 fence，core 可能在 `turn` 这次写入还没有真正生效前，就已经拿旧的 `turn` 值和 `want[j]` 一起判断是否等待。这样 Peterson 算法里“谁让谁先走”的仲裁信息就可能失效，于是仍然会出现两个 core 同时冲进临界区的情况。第二道 fence 解决的是 `turn = j` 这次写入与后续自旋读取之间的顺序问题。

6.7 Why a Single Fence Is Insufficient

只放一道 fence，只能完整堵住其中一个窗口，另一个窗口仍然可能漏掉。

如果只保留第一道 fence：

1. `want[i] = 1` 这次写入的可见性问题被修了
2. 但 `turn = j` 仍可能还没稳定可见，后面的自旋读取就先开始
3. 仲裁信息可能还是错的

如果只保留第二道 fence：

1. `turn = j` 与后续读取之间的顺序问题被修了
2. 但 `want[i] = 1` 这次“我要进入”的声明仍可能没有及时被对方看到
3. 对方还是可能读到旧的 `want[i] == 0`

所以这道题不是“随便放一个 fence 就行”，而是必须分别把 `want[i]` 和 `turn` 这两个关键写入都在进入自旋判断前排好顺序。只有两道 fence 都在，Peterson 锁需要的可见性和顺序性才同时成立，这也是为什么实测结果从无 fence 时的 `2000 / 4000` 恢复成了有 fence 时的 `4000 / 4000`。